

سیستم هشدار سریع و هوشمندی آب‌وهوای شهری (Smart Urban Weather Intelligence & Early Warning System)

چکیده و بیان مسئله

مدیریت بحران‌های شهری و کاهش خطرات ناشی از پدیده‌های جوی، نیازمند دسترسی بلادرنگ به داده‌های دقیق و پردازش سریع آن‌هاست. کلان‌شهری مانند مشهد، با توجه به تراکم جمعیت، مناطق صنعتی و ترافیک بالا، در معرض خطراتی نظیر آلودگی شدید هوا (AQI بالا)، طوفان‌های فصلی و بارش‌های ناگهانی قرار دارد. سیستم‌های سنتی پیش‌بینی آب‌وهوا معمولاً فاقد هشدارهای منطقه‌ای، بلادرنگ و کاربرمحور هستند. این پروژه با هدف رفع این خلاء، یک سیستم کاملاً خودکار و بدون نیاز به سرور (Serverless) طراحی کرده است که داده‌های هواشناسی را از رابط برنامه‌نویسی OpenWeatherMap (OWM) و داده‌های کیفیت هوا را از پایگاه World Air Quality Index (WAQI) دریافت می‌کند. سیستم با استفاده از یک موتور ارزیابی ریسک (Risk Engine)، شرایط بحرانی را شناسایی کرده و از طریق هوش مصنوعی (مدل‌های زبانی بزرگ Groq LLM)، هشدارهای شخصی‌سازی‌شده و قابل فهم را مستقیماً به پیام‌رسان تلگرام شهروندان و مدیران بحران ارسال می‌نماید.

اهداف اصلی پروژه

- ارزیابی خودکار ریسک (Automated Risk Assessment): پایش مداوم شاخص‌هایی نظیر سرعت باد، احتمال بارش و کیفیت هوا، و تشخیص شرایط بحرانی پیش از وقوع.
- معماری بدون سرور و مستقل (Zero-Dependency Serverless Architecture): اجرای زمان‌بندی‌شده‌ی فرآیندها در بستر GitHub Actions بدون نیاز به تخصیص سرور اختصاصی و با تکیه بر کتابخانه‌های استاندارد پایتون (کاهش هزینه‌های نگهداری).
- ارسال یکپارچه و قابل اطمینان پیام‌ها (Reliable Telegram Dispatch): تضمین ارسال پیام‌های هشدار با بهره‌گیری از یک سیستم مدیریت وضعیت (State Machine) جهت جلوگیری از ارسال پیام‌های تکراری (Spam).
- پردازش هوشمند و تنزل تدریجی (Graceful Degradation): مدیریت هوشمند خطاهای شبکه و خرابی سنسورهای مبدأ با استفاده از داده‌های کش‌شده و فصلی، تا سیستم تحت هیچ شرایطی از کار نیفتد.

معماری سیستم و زیرساخت

معماری این سیستم بر پایه یک خط لوله رویدادمحور (Event-Driven Pipeline) در GitHub Actions استوار است. در هر اجرای زمان‌بندی‌شده، داده‌ها از API‌های خارجی جمع‌آوری شده و به «موتور ریسک» سپرده می‌شوند. سیستم دارای یک State Machine است که خروجی را در دو مسیر کلی هدایت می‌کند: ۱. **حالت هشدار (Alert / Predictive Warning)**: در صورت وجود هشدار رسمی یا عبور شاخص‌ها از حد مجاز، یک رویداد بحرانی ثبت می‌شود. ۲. **حالت آسمان صاف (Clear Skies)**: در صورت عادی بودن شرایط، یک گزارش روزانه و خلاصه وضعیت هوشمند تولید می‌شود. در هر دو حالت، سیستم با استفاده از Groq LLM داده‌های خام عددی را به متون روان، ساختاریافته و دارای ایموژی‌های بصری تبدیل می‌کند که برای کاربر نهایی کاملاً قابل درک است.

بخش‌های کلیدی کد با توضیحات

قطعه‌کد اول: موتور ارزیابی ریسک پیش‌بینانه (The Predictive Risk Engine)

این بخش از کد مسئول تحلیل داده‌های ترکیب‌شده‌ی OWM و WAQI است. به جای تکیه صرف بر هشدارهای رسمی، سیستم دارای یک هوش پیش‌بینانه است که در صورت وزش باد شدید (بیش از ۱۵ متر بر ثانیه)، احتمال بارش بالا (بیش از ۷۰ درصد) یا کیفیت هوای خطرناک (AQI بالای ۱۵۰)، به صورت مستقل اعلام وضعیت هشدار می‌کند. این ویژگی برای واکنش سریع در برابر آلودگی‌های ناگهانی هوا بسیار حیاتی است.

```
# Analysis & State determination based on environmental thresholds
if current_alerts:
    state_enum = PipelineState.ALERT
elif global_metrics["max_wind"] > 15 or global_metrics["max_pop"] > 0.70 or
global_metrics["aqi"] > 150:
    state_enum = PipelineState.PREDICTIVE_WARNING

if state_enum == PipelineState.PREDICTIVE_WARNING:
    now_ts = int(datetime.now(timezone.utc).timestamp())

    risk_reasons = []
    # Identify specific triggers for the predictive warning
    if global_metrics['max_wind'] > 15:
        risk_reasons.append(f"Wind {global_metrics['max_wind']}m/s")
    if global_metrics['max_pop'] > 0.70:
        risk_reasons.append(f"Precipitation Prob
{global_metrics['max_pop']*100}%")
    if global_metrics['aqi'] > 150:
        risk_reasons.append(f"Hazardous Air Quality (AQI
{global_metrics['aqi']})")

    alert_payload = {
        "event": "Predictive Warning",
        "start": now_ts,
        "end": now_ts + (4 * 3600),
        "description": f"Predictive Engine detected high risk conditions:
{' '.join(risk_reasons)}.",
        "max_pop": global_metrics["max_pop"],
        "zones": ["All Mashhad Zones"]
    }
```

قطعه‌کد دوم: ماشین وضعیت و مکانیزم هشینگ (The State Machine / Hashing Mechanism)

برای جلوگیری از ارسال اخطارهای تکراری به کاربران (Spam) ناشی از اجراهای متوالی GitHub Actions، سیستم برای هر رویداد یک شناسه یکتا (Hash) تولید می‌کند. همان‌طور که در معماری سیستم تصمیم‌گیری شده است، متغیر sender_name از تولید این Hash حذف شده است. این امر قابلیت Idempotency ایجاد کرده و تضمین می‌کند که اگر چندین آژانس هواشناسی یک هشدار واحد را صادر کنند، سیستم همه را به عنوان یک رویداد واحد پردازش کند.

```
def alert_id_for(alert):  
    """  
    Architecture Note for Academic Review:  
    The 'sender_name' was intentionally omitted from the cryptographic hash  
    generation.  
    This decision enforces automatic deduplication (Idempotency). If  
    multiple agencies  
    (e.g., local vs. national meteorological organizations) issue redundant  
    warnings for  
    the exact same event at the exact same start time, the system will  
    resolve them to  
    a single unique hash, preventing notification spam.  
    """  
    # Create a unique MD5 hash based strictly on the event type and its  
    start time  
    raw = f"{alert.get('event')}|{alert.get('start')}}"  
    return hashlib.md5(raw.encode("utf-8")).hexdigest()
```

قطعه‌کد سوم: منطق تنزل تدریجی و پایداری شبکه (The Fallback / Graceful Degradation Logic)

در سامانه‌های حیاتی و Production-Grade، خطای شبکه‌ای یک سرویس خارجی نباید منجر به از کار افتادن کل سیستم شود. این قطعه‌کد نشان می‌دهد که چگونه سیستم در صورت قطعی یا عدم پاسخگویی API مبدأ، از بروز خطاهای زنجیره‌ای جلوگیری کرده و مقادیر پایه فصلی (Fallback) را جایگزین می‌کند. سپس با ثبت کد وضعیت خطا، به لایه‌های بالاتر (LLM) اطلاع می‌دهد تا رابط کاربری (UI) خود را متناسب با این شرایط تنزل یافته تنظیم کند.

```
def fetch_weather_data_for_zone(lat, lon, api_key, timeout=10):  
    # Prepare API request parameters  
    params = {  
        "lat": lat,  
        "lon": lon,  
        "appid": api_key,  
        "exclude": "minutely",  
        "units": "metric",  
        "lang": "fa",  
    }  
    url = f"{OWM_BASE_URL}?{urllib.parse.urlencode(params)}"
```

```

request = urllib.request.Request(url, headers={"User-Agent": "weather-
alert-bot/1.0"})

# Define a realistic baseline fallback payload for Mashhad during API
outages
fallback = {
    "current": {"temp": 20.0, "humidity": 30, "wind_speed": 5.0, "uvi":
5.0},
    "hourly": [{"dt": 0, "temp": 20.0, "pop": 0.0, "wind_speed": 5.0,
"uvi": 5.0} for _ in range(24)],
    "alerts": []
}

try:
    # Attempt to fetch real-time data
    with urllib.request.urlopen(request, timeout=timeout) as response:
        payload = json.loads(response.read().decode("utf-8"))
        return payload, str(response.getcode())
except urllib.error.HTTPError as e:
    # Gracefully degrade and return fallback data upon HTTP error
    print(f"[OWM] HTTP {e.code} {e.reason} - lat={lat} lon={lon}",
file=sys.stderr)
    return fallback, f"HTTP {e.code}"
except Exception as e:
    # Catch all other network errors to prevent pipeline crashes
    print(f"[OWM] Unexpected error - {e} - lat={lat} lon={lon}",
file=sys.stderr)
    return fallback, "Error"

```

نتیجه‌گیری و چشم‌انداز

پروژه «سیستم هشدار سریع آب‌وهوای شهری»، نمونه‌ای بارز از ترکیب مهندسی نرم‌افزار نوین، پایداری سیستم (Reliability) و هوش مصنوعی است. این سامانه با اتخاذ معماری Serverless و پیاده‌سازی مکانیزم‌های پیشرفته‌ای چون Graceful Degradation و Idempotency، نه تنها از قابلیت اطمینان بالایی برخوردار است، بلکه به دلیل عدم وابستگی به زیرساخت‌های پرهزینه، مدلی ایده‌آل برای مقیاس‌پذیری در سایر کلان‌شهرها محسوب می‌شود. در آینده می‌توان با اتصال سنسورهای محلی اینترنت اشیا (IoT) به این خط لوله، دقت مکانی و زمانی این هشدارها را به شکل چشم‌گیری ارتقاء داد.